

Fundamental Algorithm Techniques – Problem Set #9

Ready-to-copy solutions

Problem 1 — Finite functions on a computer

We consider finite functions of the form

$$\begin{aligned} &[\\ F : \{0,1\}^n &\rightarrow A \\ &] \end{aligned}$$

where $|\{0,1\}^n| = 2^n$.

1) Output $\{0,1\}$

For each of the (2^n) inputs, the function can output **either 0 or 1**.

- Number of choices per input: **2**
- Number of inputs: **(2^n)**

$$\begin{aligned} &[\\ \boxed{\text{Number of functions}} &= 2^{2^n} \\ &] \end{aligned}$$

2) Output $\{-1,0,1\}$

For each of the (2^n) inputs, the output has **3 possible values**.

- Number of choices per input: **3**
- Number of inputs: **(2^n)**

$$\begin{aligned} &[\\ \boxed{\text{Number of functions}} &= 3^{2^n} \\ &] \end{aligned}$$

3) Output $\{0,1\}^m$

Each output is an (m) -bit vector.

- Number of possible outputs: **(2^m)**
- Number of inputs: **(2^n)**

$$\begin{aligned} &[\\ \boxed{\text{Number of functions}} &= (2^m)^{2^n} = 2^{m \cdot 2^n} \\ &] \end{aligned}$$

Decision tree intuition (optional explanation)

A decision tree has:

- (2^n) leaves (one per input)
- each leaf labeled by one possible output

Number of functions = number of ways to label the leaves.

Problem 2 — NAND equivalence (4/10 pts)

NAND definition

$$\begin{aligned} &[\\ \text{NAND}(A,B) &= \neg(A \wedge B) \\ &] \end{aligned}$$

$$A \quad B \quad A \text{ NAND } B$$

0 0 1

0 1 1

1 0 1

1 1 0

1) NOT using NAND

[
$$\neg A = A \text{ NAND } A$$

]

2) AND using NAND

[
$$A \text{ AND } B = (A \text{ NAND } B) \text{ NAND } (A \text{ NAND } B)$$

]

3) OR using NAND (De Morgan)

[
$$A \text{ OR } B = (A \text{ NAND } A) \text{ NAND } (B \text{ NAND } B)$$

]

Conclusion

Since **AND**, **OR**, and **NOT** can be built using only NAND,

[
$$\text{NAND is a universal gate}$$

]

Problem 3 — Universality of Boolean Circuits (3/10 pts)

We consider:

[
$$F : \{0,1\}^n \rightarrow \{0,1\}$$

]

Step 1: Function as a truth table

- There are (2^n) possible inputs
- For each input, the function outputs either 0 or 1

So **any Boolean function** can be described by its truth table.

Step 2: Build the function using DNF

For every input where $(F(x) = 1)$:

- Create a conjunction (AND) of literals
- Combine all such terms using OR

This is **Disjunctive Normal Form (DNF)**.

Step 3: Circuit size

- Each term uses at most (n) gates
- There are at most (2^n) terms

$$\boxed{\text{Circuit size} = O(n \cdot 2^n)}$$

Final conclusion

Every Boolean function:

- Can be implemented by a Boolean circuit
- With size $(O(n \cdot 2^n))$

$$\boxed{F \text{ is computable}}$$